

Linux

Introduction to Linux

Linux is an open-source operating system based on the Linux kernel. It was originally developed by Linus Torvalds in 1991. Linux is widely used in servers, cloud computing, software development, networking, cybersecurity, embedded systems, and many other areas of information technology.

Unlike proprietary operating systems, Linux provides users with access to its source code, allowing developers and organizations to modify, customize, and distribute the operating system according to their requirements. Linux is known for its stability, security, flexibility, and efficient use of system resources.

Linux Kernel

The Linux kernel is the core component of a Linux operating system. It acts as an interface between the hardware and software. The kernel manages important system resources such as the processor, memory, storage devices, and input/output devices.

It also manages processes and provides security and access-control mechanisms. Users normally interact with the kernel indirectly through the shell and other system utilities.

Linux Distributions

A complete Linux operating system is generally provided as a Linux distribution, commonly called a distro. A distribution contains the Linux kernel along with system utilities, libraries, package managers, and applications.

Some commonly used Linux distributions are:

- **Ubuntu:** A user-friendly distribution commonly used for development and general-purpose computing.
- **Debian:** A stable distribution widely used for servers and other applications.
- **Fedora:** A modern distribution often used by developers and technology enthusiasts.
- **Kali Linux:** A Linux distribution designed for cybersecurity and security testing.
- **Arch Linux:** A highly customizable distribution intended for users who want greater control over their system.

Important Features of Linux

Linux provides several features that make it suitable for professional and technical environments:

1. **Open Source:** The source code is available for study, modification, and distribution.

2. **Security:** Linux uses user accounts, file permissions, and access-control mechanisms to protect system resources.
3. **Stability:** Linux systems can operate for long periods with minimal interruption.
4. **Multituser Support:** Multiple users can use system resources while maintaining separate permissions.
5. **Multitasking:** Linux can execute multiple processes simultaneously.
6. **Customization:** Users can customize the operating system, desktop environment, shell, and system configuration.
7. **Command-Line Interface:** Linux provides a powerful terminal for managing files, processes, networks, and system configurations.

Linux File System

Linux uses a hierarchical file system in which files and directories are organized from a single root directory represented by /.

Some important directories include:

- /home – Contains users' personal files and directories.
- /etc – Contains system configuration files.
- /bin – Contains essential user commands.
- /var – Contains variable data such as logs.
- /tmp – Used for temporary files.
- /root – Home directory of the root user.
- /usr – Contains many user applications and system resources.

Understanding the Linux file system is important for managing files and configuring the operating system.

Linux Shell and Terminal

The shell provides a command-line interface through which users can interact with Linux. It accepts commands from the user and communicates with the operating system to perform the requested operations.

Common Linux shells include **Bash, Zsh, and Fish**. Bash is one of the most commonly used shells. The terminal can be used to perform tasks such as creating files, navigating directories, installing software, managing processes, and configuring network settings.

Basic Linux Commands

Some commonly used Linux commands are:

Command Purpose

pwd	Displays the current working directory
ls	Lists files and directories
cd	Changes the current directory
mkdir	Creates a new directory
touch	Creates a new file
cat	Displays the contents of a file
cp	Copies files or directories
mv	Moves or renames files
rm	Removes files or directories
echo	Displays text or writes text to a file
clear	Clears the terminal screen
sudo	Executes a command with elevated privileges

For example, the following commands can be used to create a directory and a file:

```
mkdir practice
```

```
cd practice
```

```
touch test.txt
```

```
echo "Hello Linux" > test.txt
```

```
cat test.txt
```

These commands create a directory named practice, enter the directory, create a file named test.txt, write text into the file, and display its contents.

Linux Permissions

Linux uses a permission system to control access to files and directories. Permissions determine whether a user can **read, write, or execute** a particular file.

The three basic permissions are:

- **r (Read):** Allows the contents of a file to be viewed.
- **w (Write):** Allows the contents of a file to be modified.
- **x (Execute):** Allows a file to be executed as a program.

Permissions are generally assigned to three categories:

- **User (u):** The owner of the file.
- **Group (g):** Users belonging to the file's group.
- **Others (o):** All other users.

The chmod command can be used to modify permissions.

Linux in Cybersecurity

Linux plays an important role in cybersecurity because of its flexibility, command-line capabilities, networking tools, and availability of security-focused software.

Kali Linux is a popular Linux distribution used for cybersecurity training and authorized security testing. It provides various tools for activities such as network analysis, vulnerability assessment, digital forensics, and penetration testing.

For example, **Nmap** can be used to discover hosts and analyze network services in an authorized environment. Other security tools can be used for vulnerability assessment and security analysis.

Advantages of Linux

The major advantages of Linux include:

- Free and open-source software
- Strong security and permission management
- High stability and reliability
- Efficient resource utilization
- Extensive command-line capabilities
- High level of customization
- Wide support for programming and development tools
- Strong presence in servers and cloud environments

DevOps

What is DevOps?

DevOps is a combination of **software Development (Dev)** and **IT Operations (Ops)** practices, culture, and tools. It focuses on improving collaboration between development and operations teams and automating the software delivery process.

The main objectives of DevOps are:

- Shortening the software development lifecycle.
- Delivering features, fixes, and updates frequently.
- Improving software reliability.
- Automating repetitive tasks.
- Improving collaboration between development, operations, and QA teams.
- Continuously improving software delivery through feedback.

DevOps is not a single tool or job title. It represents a cultural shift based on **collaboration, automation, and continuous improvement**.

Importance of DevOps

DevOps is important because it provides:

- **Faster delivery:** New features and fixes can be released more quickly.

- **Better collaboration:** Development, operations, and QA teams work together.
- **Improved quality:** Automated testing helps detect problems early.
- **Reliable deployments:** Automation reduces manual errors.
- **Faster recovery:** Problems can be identified and resolved more quickly.

DevOps is commonly used for cloud-native applications, startups, enterprise IT systems, and SaaS products.

Origin and Evolution of DevOps

Before DevOps, development and operations teams generally worked separately. Developers focused on writing software, while operations teams were responsible for deploying and maintaining it.

This separation often resulted in:

- Communication problems
- Deployment delays
- Misunderstandings
- Integration problems
- Difficulties when releases failed

The **Agile movement** improved software development practices, but the gap between development and operations still existed.

The term **DevOps** was popularized by **Patrick Debois**, who organized the first DevOpsDays conference in Belgium in 2009. DevOps was influenced by Agile, Lean principles, and Google's Site Reliability Engineering practices.

Evolution Timeline

- **2001:** Agile Manifesto published.
- **2007–2008:** Increasing discussion about the gap between development and operations.
- **2009:** First DevOpsDays conference.
- **2010s:** Tools such as Jenkins, Docker, and Ansible accelerated DevOps adoption.
- **2013:** Docker was released and helped popularize containerization.
- **Today:** DevOps is widely used in startups and enterprises.

Core Principles of DevOps

The major principles of DevOps are:

Collaboration

Development, operations, and QA teams work together instead of operating in separate silos.

Automation

Repetitive tasks such as:

- Building
- Testing
- Deployment
- Infrastructure changes

are automated.

Continuous Integration and Delivery

Small changes are integrated frequently and made ready for release through automated processes.

Monitoring and Feedback

Systems are continuously monitored, and the information collected is used to improve the software and infrastructure.

Infrastructure as Code

Infrastructure is managed through version-controlled configuration files instead of relying entirely on manual setup.

DevOps Lifecycle

The DevOps lifecycle is a continuous cycle rather than a one-time process.

The major stages are:

Plan → Code → Build → Test → Release → Deploy → Operate → Monitor → Plan

Plan

Requirements, features, and priorities for the next release are defined.

Code

Developers write and review application source code.

Build

The source code is compiled and packaged into deployable artifacts.

Test

Automated tests are performed to detect bugs and problems.

Release

A tested build is prepared for deployment.

Deploy

The release is deployed to staging or production environments.

Operate

The application is run and maintained in production.

Monitor

The application and infrastructure are monitored for performance, errors, and usage. The collected information provides feedback for the next planning stage.

Continuous Integration and Continuous Delivery

Continuous Integration

Continuous Integration (CI) is the practice of frequently integrating code changes into a shared repository.

Whenever code is changed:

1. The change is pushed to the repository.
2. A build is automatically triggered.
3. Automated tests are executed.
4. Problems are detected early.

This reduces the chance of integration problems becoming larger issues.

Continuous Delivery

In **Continuous Delivery**, every change that passes automated tests is kept ready for production release. However, production deployment requires manual approval.

Continuous Deployment

In **Continuous Deployment**, changes that successfully pass the automated pipeline are automatically deployed to production without a manual approval step.

Difference

Continuous Delivery	Continuous Deployment
Release is ready	Release is automatically deployed
Manual approval may be required	No manual deployment step
Production deployment is controlled	Production deployment is automated

CI/CD Pipeline

A typical CI/CD pipeline consists of several stages:

Source

The latest source code is obtained from a version-control system such as Git.

Build

The application and its dependencies are compiled or prepared.

Test

Automated unit, integration, and other tests are executed.

Package

A deployable artifact is created. This can be a Docker image.

Deploy

The artifact is deployed to a staging or production environment.

A simplified pipeline can be represented as:

Source → Build → Test → Package → Deploy

Containerization and Docker

Containerization

Containerization packages an application along with everything required to run it, including:

- Application code
- Runtime
- Libraries
- Configuration

into an isolated unit called a **container**.

Containers share the host operating system's kernel, making them lightweight and faster to start than traditional virtual machines. They also help maintain consistency across development, testing, and production environments.

Docker

Docker is an open-source platform used to build, ship, and run containerized applications.

Docker made containerization easier to use through simple commands and a broad ecosystem of tools.

Benefits of Docker in DevOps

Consistency

The same application environment can be used across different stages, reducing the "**works on my machine**" problem.

Speed

Containers start much faster than traditional virtual machines.

Portability

A Docker image can be used on a developer's computer, a testing server, or a cloud environment.

CI/CD Integration

An application can be built as an image and then deployed using the same image throughout the pipeline.

Resource Efficiency

Containers share the host operating system kernel and generally require fewer resources than separate virtual machines.

Docker vs Virtual Machines

Aspect	Virtual Machine	Docker Container
Operating System	Each VM has a complete guest OS	Containers share the host OS kernel
Boot Time	Usually minutes	Usually seconds
Size	Often gigabytes	Typically megabytes
Resource Usage	Higher	Lower
Isolation	Strong hardware-level isolation	Process-level isolation

Docker Architecture

Docker follows a client-server architecture.

Docker Client

The docker command-line tool used by users to interact with Docker.

Docker Daemon

The dockerd background service that builds, runs, and manages containers.

Docker Image

A read-only template used to create containers.

Docker Container

A running instance of a Docker image.

Docker Registry

A system used to store and distribute Docker images. **Docker Hub** is an example.

Docker's Underlying Technologies

Docker uses several Linux technologies:

Namespaces

Namespaces isolate a container's view of:

- Processes
- Networking
- File systems

Control Groups — cgroups

Cgroups control and monitor resources such as:

- CPU
- Memory

used by containers.

Union File Systems

These allow Docker images to be built using reusable layers, helping reduce storage requirements and build time.

Dockerfile

A **Dockerfile** is a text file containing instructions used to automatically build a Docker image.

Example:

```
FROM node:18-alpine
```

```
WORKDIR /app
```

```
COPY package.json .
```

```
RUN npm install
```

```
COPY . .
```

```
EXPOSE 3000
```

```
CMD ["npm", "start"]
```

Important Dockerfile instructions include:

Instruction	Purpose
FROM	Specifies the base image
WORKDIR	Sets the working directory
COPY / ADD	Copies files into the image
RUN	Executes a command during image creation
EXPOSE	Documents the port used by the application
CMD / ENTRYPOINT	Defines the command executed when the container starts

Basic Docker Commands

Some important Docker commands are:

```
docker build -t name .
```

Builds an image from a Dockerfile.

```
docker images
```

Lists locally available images.

```
docker run -d -p 8080:80 name
```

Runs a container and maps a host port to a container port.

```
docker ps
```

Lists running containers.

```
docker stop <id>
```

Stops a running container.

```
docker exec -it <id> bash
```

Opens an interactive shell inside a running container.

```
docker logs <id>
```

Displays container logs.

Docker in CI/CD Workflow

Docker can be integrated into CI/CD as follows:

Build → Test → Push → Deploy

1. The pipeline builds a Docker image.
2. Automated tests are performed using the image.
3. The tested image is pushed to a container registry.
4. The same image is deployed to staging and production.

Using the same tested image helps maintain consistency between environments.

Docker Compose

Docker Compose is used to define and run multi-container applications through a YAML configuration file.

For example, an application may consist of:

- Web application
- Database

A Compose file can define both services and their configurations.

Commands include:

`docker-compose up -d`

to start the services, and:

`docker-compose down`

to stop them.

Container Orchestration

When applications use hundreds of containers across multiple machines, manually managing them becomes difficult.

Container orchestration automates:

- Deployment
- Scaling
- Networking
- Recovery

Two orchestration technologies mentioned in the document are:

- Kubernetes
- Docker Swarm

Kubernetes

Kubernetes is a container orchestration platform.

Important concepts include:

Pod

The smallest deployable unit in Kubernetes. It can contain one or more containers.

Node

A physical or virtual machine that runs Pods.

Cluster

A collection of nodes managed together.

Service

Provides a stable network endpoint for a group of Pods.

Deployment

Manages the desired state and scaling of Pods.

Docker Swarm

Docker Swarm is Docker's built-in orchestration technology.

It can:

- Cluster multiple Docker hosts
- Schedule containers
- Manage containers across hosts

The document describes it as having a simpler learning curve than Kubernetes for smaller-scale deployments.

Infrastructure as Code

Infrastructure as Code (IaC) means managing and provisioning infrastructure using machine-readable configuration files instead of configuring infrastructure manually.

Infrastructure can include:

- Servers
- Networks
- Cloud resources
- Other infrastructure components

IaC makes infrastructure:

- Versioned
- Reviewable
- Repeatable
- Consistent

Benefits of IaC

Consistency: The same configuration can produce identical environments.

Speed: New environments can be created quickly.

Version Control: Infrastructure changes can be tracked, reviewed, and rolled back.

Popular IaC Tools

Tool	Purpose
Terraform	Provisions and manages infrastructure across cloud providers
Ansible	Configuration management and application deployment
AWS CloudFormation	Provisions AWS resources
Pulumi	Defines infrastructure using general-purpose programming languages

Monitoring and Logging

Monitoring provides visibility into the performance of applications and infrastructure.

It helps detect problems early and can reduce downtime.

Important Metrics

Metric	Purpose
Availability	Measures whether a service is accessible
Latency	Measures how quickly requests are processed
Error Rate	Shows failures requiring attention
Resource Usage	Tracks CPU, memory, and disk usage

Monitoring and Logging Tools

- **Prometheus:** Collects time-series metrics.
- **Grafana:** Creates dashboards for visualizing metrics.
- **ELK Stack:** Used for collecting, searching, and visualizing log data.
- **Datadog:** Commercial monitoring and analytics platform.

Popular DevOps Tools

Category	Tools
Version Control	Git, GitHub, GitLab, Bitbucket
CI/CD	Jenkins, GitHub Actions, GitLab CI, CircleCI
Containerization	Docker, Podman
Orchestration	Kubernetes, Docker Swarm

Category	Tools
Configuration Management	Ansible, Chef, Puppet
Infrastructure as Code	Terraform, Pulumi, CloudFormation
Monitoring & Logging	Prometheus, Grafana, ELK Stack, Datadog
